



同济大学软件学院

School of Software Engineering, Tongji University



Software Testing Foundation Level

Chapter 6: Tool Support for Testing

刘琴 *Zin Liu*



Software Testing Foundation Level

- **1 Fundamental of testing**
- **2 Testing throughout the life cycle**
- **3 Static techniques**
- **4 Test Design Techniques**
 - [Extension]: Risk Based Testing Analysis and Design
 - [Extension]: Usability Testing
- **5 Test management**
- **6 Tool support for testing**
 - [Extension]: JUnit/Selenium



6. Tool Support for Testing

1. Types of test tools
2. Effective use of tools, potential benefits and risks
3. Introducing a tool into an organization

Test Engineering Foundation



Essential Knowledge for Test Professionals

Chapter 6: Tool Support for Testing

Section 1: Types of test tools



Types of Test Tools

- Key concepts
 - Different types of test tools
 - Programmers' test tools

■ Terms to remember



Test Tool Classification

- Tools can be classified by **the primary testing activities** that they support
- For most classifications of tool, there are both commercial and freeware options available
- Tools **automate repetitive tasks**, make testing more efficient
- Tools **improve the reliability of testing**; e.g., automating large data comparisons, generating load
- Tools are either intrusive or non-intrusive
 - Intrusive tools have probe effects
 - Non-intrusive tools are generally more expensive
- Some tools are primarily for developers



Test Tools

1. Test Management Tools
2. Requirements Management Tools
3. Bug Tracking Tools
4. Configuration Management Tools
5. Review Process Support Tools
6. Static Analysis Tools
7. Modeling and Design Tools
8. Test Design Tools
9. Test Data Tools
10. Test Execution Tools
11. Test Frameworks, Harnesses, Simulators
12. Test Comparators
13. Coverage Measurement Tools
14. Security Tools
15. Performance, Monitoring, Dynamic Analysis
16. Other Tools



Recap: Test Suites in Omninet

Activities:

- *Lifecycle (V or Iterative?)*
- *Review (early testing)*
- *Project Nature (User-Centred)*

repetitive tasks:

improve the reliability of testing:

- *Large amount of data*

primarily used by Developer Tools:

- *Debugging tools*
- *Static analysis tools*
- *Modeling and design tools*
- *Code coverage measurements tools etc.,*

1. Kiosk functionality
2. Call center functionality
3. **Data** center functionality
4. End-to-end functionality
5. **Security** and privacy
6. Reliability
7. Supportability
8. Localization
9. **Performance**
10. Error handling/Usability
11. Exploratory
12. Beta test

Test Engineering Foundation



Essential Knowledge for Test Professionals

Chapter 6: Tool Support for Testing

Section 1:
Exercise





Exercise: Omninet Test Tools

- List the types of tools discussed in this section which you think would be useful for testing Omninet.
- Discuss.



1. Test Management Tools

- Provide traceability of tests, test results and incidents to the test basis
- Allow logging of test results, and generation of progress reports
- Help manage tests and testing
- Interface to test execution, bug tracking, and requirement management tools
- Provide version control or interface with an external configuration management tool
- Perform quantitative analysis (metrics) related to the tests
- **SOTA Tool Examples**
- Jira + Xray / Zephyr, TestRail, qTest, PractiTest, Polarion

1.1 Test Management Tools

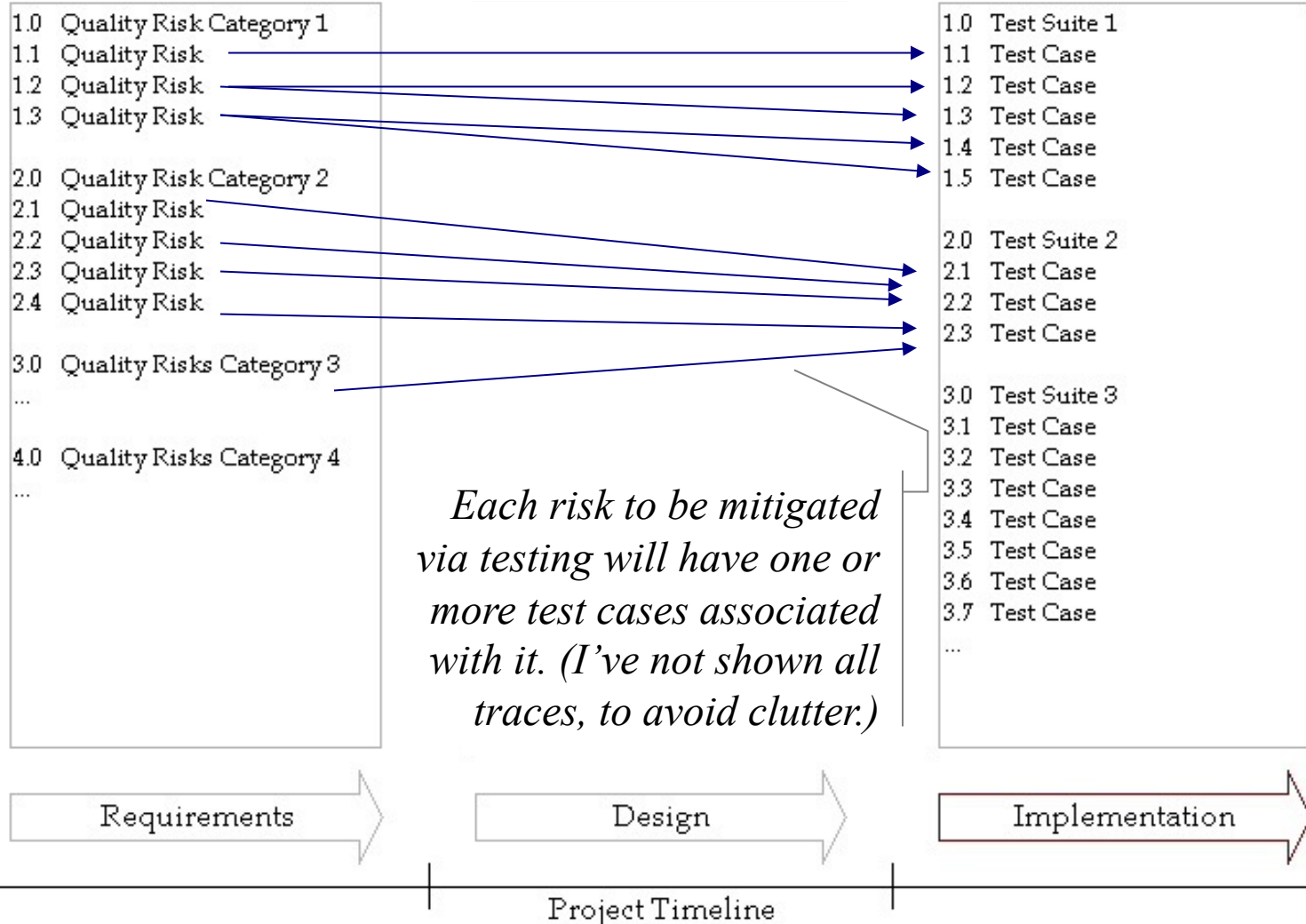
Quality Risk Analysis

High-Level Test Design

Low-Level Test Design

manage tests

Testing Timeline

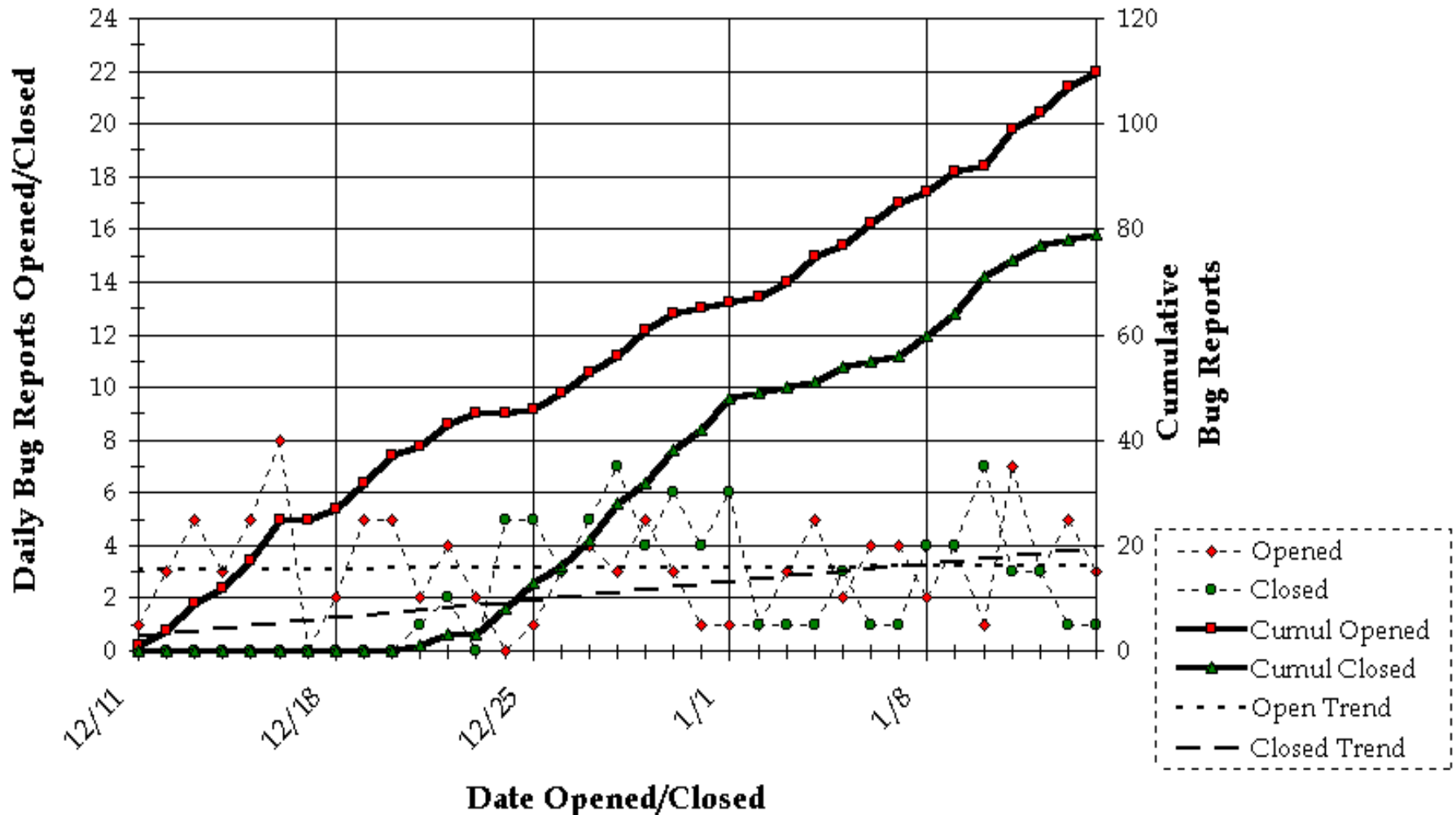


1.2 Test Management Tools -

Speedy Writer System Test

Bug Reports Opened and Closed

quantitative analysis

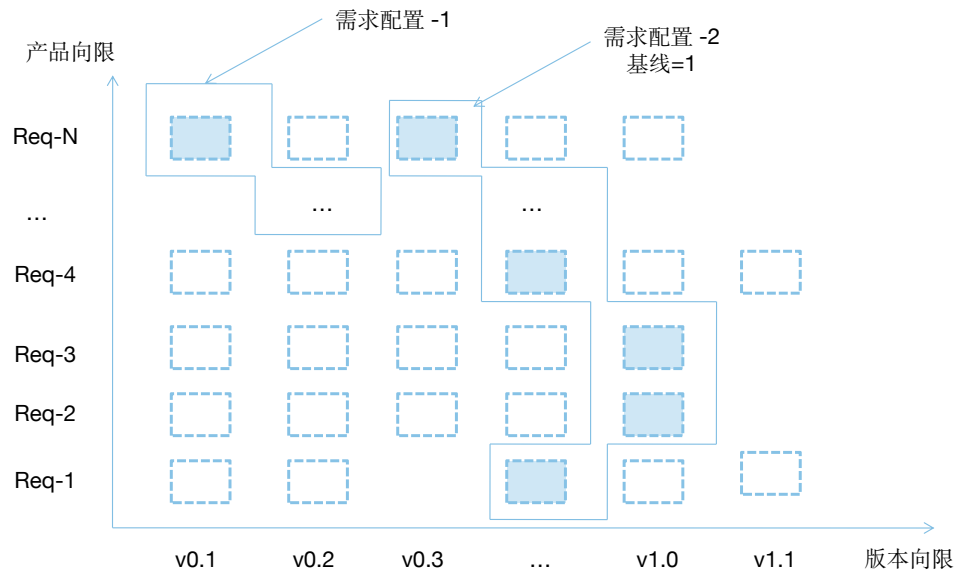
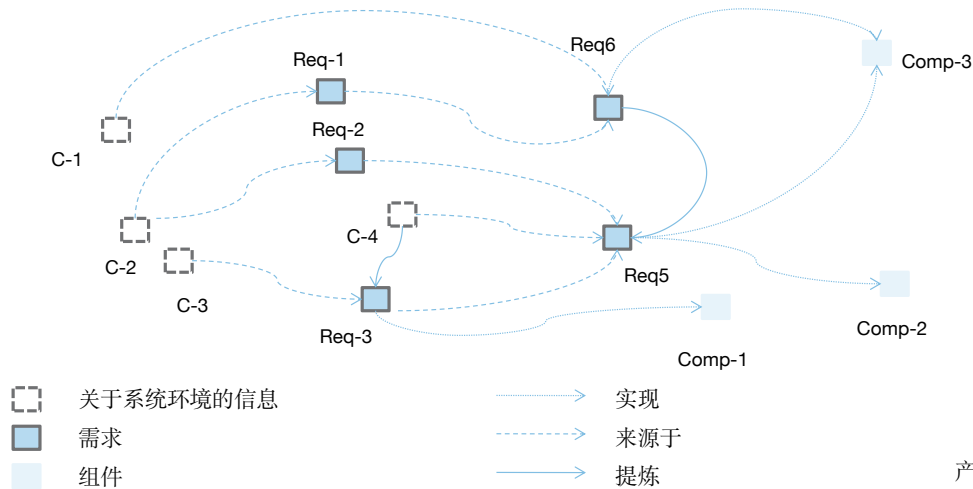




2. Requirements Management Tools

- Store requirements statements
- Check for consistency and undefined (missing) requirements
- Allow requirements to be prioritized
- Enable individual tests to be traceable to (and reported in terms of) requirements, functions, and/or features
- **SOTA Tool Examples**
- Jama Connect, IBM DOORS Next, ReQtest, Accompa, Helix RM

2. Requirements Management Tools

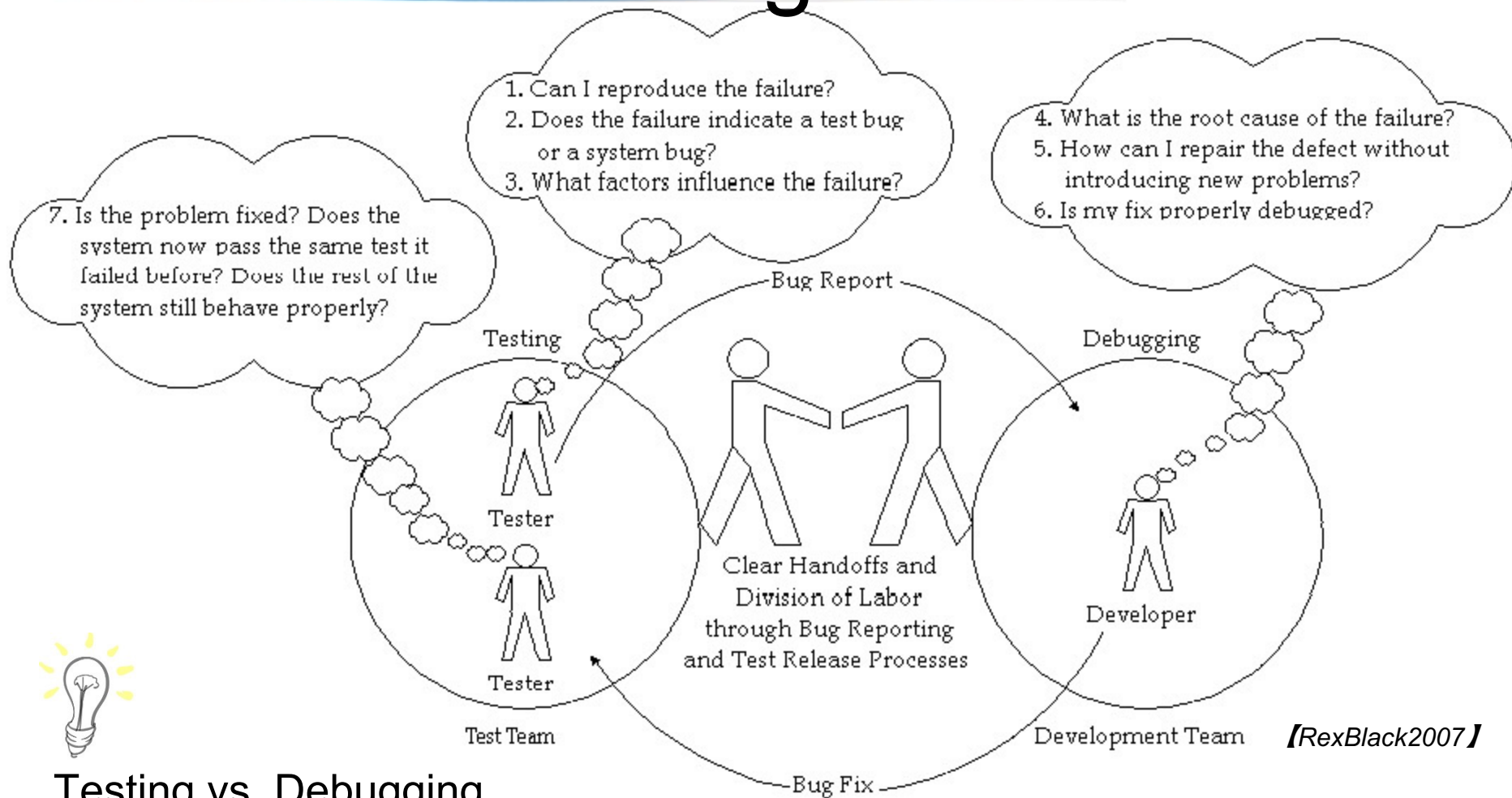


3. Bug (or Defect or Incident) Tracking Tools



- Store and manage bug reports
- Facilitate bug prioritization/ classification
- Provide state-based workflow, including assignment of actions to people (e.g., fix, confirmation test, etc.),
- Enable monitoring a project's bugs and bug status over time, especially vis-à-vis trends
- Provide support for statistical analysis (often through export; e.g., to Excel)
- Create reports and graphs (though often of limited usefulness due to the variation in needs)
- **SOTA Tool Examples**
- Jira, Bugzilla, MantisBT, YouTrack, Azure DevOps Boards

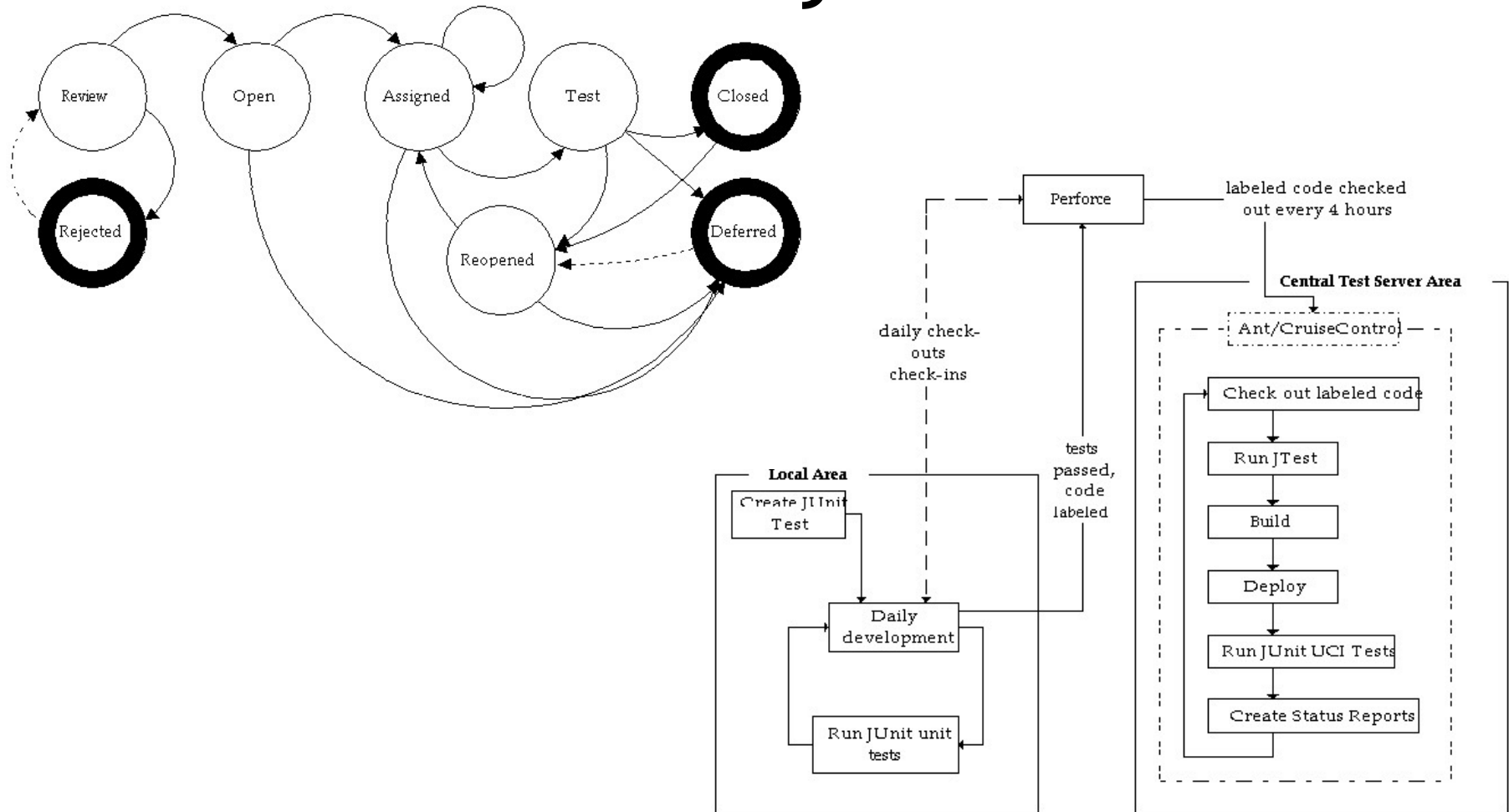
3. Bug (or Defect or Incident) Tracking Tools



Testing vs. Debugging

【RexBlack2007】

3. Bug (or Defect or Incident) Tracking Tools





4. Configuration Management Tools

- Store information about versions and builds of software and testware
- Enable traceability between testware and software work products and product variants
- Help with developing and testing on multiple configurations of hardware/ software environments
- **SOTA Tool Examples**
- Git / GitHub / GitLab, Subversion, Mercurial, Perforce Helix Core, Plastic SCM

5. Review Process Support Tools



- Store information about review processes
- Store and communicate review comments
- Report on defects and effort
- Manage references to review rules and/or checklists
- Provide aid for online reviews
- Track traceability between documents and source code
- **SOTA Tool Examples**
- Gerrit, Review Board, Crucible, Phabricator, GitHub Pull Requests + Reviewable



6. Static Analysis Tools

- Primarily used by developers
- Find defects before dynamic testing
- Enforce coding standards
- Analyze structures and dependencies
- Aid in understanding source code
- Calculate metrics from the code
- **SOTA Tool Examples**
- SonarQube, ESLint (JavaScript), Checkstyle (Java), Pylint (Python), Clang Static Analyzer (C/C++)



7. Modeling and Design Tools

- Primarily used by developers
- Help create models of the system
- Validate models
- Aid in generating some test cases based on the model
- **SOTA Tool Examples**
- Enterprise Architect, MagicDraw / Cameo, IBM Rational Rhapsody, Visual Paradigm, PlantUML



8. Test Design Tools

- Generate test inputs or the actual tests from:
 - Requirements
 - Graphical user interface
 - Design models
 - Code
- Generate expected results (though the reliability of such test oracles is often limited)
- Generate test frameworks, templates, and stubs
- **SOTA Tool Examples**
- Tricentis Tosca, TestCraft (AI), Conformiq, Curiosity (AI), Rancore



9. Test Data Tools

- Manipulate or create databases, files or data for use during test execution
- Create large volumes of useful test data
- Validate test data according to specific rules
- Analyze the data for frequency of conditions, etc.
- Scramble or anonymize live or customer data
- **SOTA Tool Examples**
- Mockaroo, Delphix, GenRocket, CA Test Data Manager, IBM Infosphere Optim



10. Test Execution Tools

- Run tests (i.e., submit inputs per an automated script and compare actual and expected results)
 - Properly used, the script is an input-independent test procedure that allows you to repeat tests with different data, run similar tests, and easily maintain the tests
 - Improperly used--e.g, typical capture-playback--the scripts are intertwined with the data and/or expected results, have hard-coded delays in them, and are quite fragile
 - Capture-playback can be useful during exploratory testing
- Include comparators
- Produce analyzable logs
- Work at GUI, API, or CLI
- **SOTA Tool Examples**
- Selenium (Web), Appium (Mobile), Postman / Newman (API), Cypress (Modern Web), Playwright (Cross-browser)



11. Test Frameworks, Harnesses, Simulators

- Replace or substitute for missing and/or potentially troublesome pieces of hardware or software
- Facilitate testing of unit(s) by generating and/or supporting drivers, stubs, and/or mock objects that replace portions of the system, which are unavailable or removed to isolate the unit
- Provide execution frameworks in middleware to test languages, operating systems or hardware
- **SOTA Tool Examples**
- JUnit / TestNG (Java), Mockito / EasyMock, PyTest (Python), WireMock (HTTP Simulation), Hoverfly (API Simulation)



12. Test Comparators

- Check files, databases, or test results against expectations, either during test execution or afterward
- May include an automated test oracle rather than comparing against static baselines
- Can be made smart by programming it to handle dynamic variables like dates and time, or to ignore the order of records when sort-orders in reports or queries are ambiguous
- **SOTA Tool Examples**
- XMLUnit, DbUnit, ApprovalTests, OCUit (Objective-C), Examine

13. Coverage Measurement Tools



- Primarily used by developers
- Either intrusive or non-intrusive
- Measure the percentage of specific types of code structure that have been exercised
 - Statements
 - Branches or decisions
 - Objects
 - Function calls
- Check how thoroughly a set of tests has executed the measured type of structure
- **SOTA Tool Examples**
- JaCoCo (Java), Gcov / LCOV (C/C++), Coverage.py (Python), Istanbul (JavaScript), OpenCover (.NET)



14. Security Tools

- Check for computer viruses
- Simulate various kinds of attacks
- Simulate various security conditions
- Check code for known security violations
- **SOTA Tool Examples**
- OWASP ZAP, Burp Suite, SonarQube (Security Plugin), Trivy / Clair (Container), Snyk (Dependency)

15. Performance, Monitoring,



Dynamic Analysis

- Primarily used by developers, though many testers use performance and monitoring tools
- Dynamic analysis tools often are used to check for time dependencies or memory leaks
- Monitor and report on how a system behaves under simulated usage conditions
- Generate various (hopefully realistic) load conditions for the application, a database, network or server, often per some script or programmed procedure
- Monitor, analyze, verify and report on usage of specific system resources, and give warnings of possible problems
- **SOTA Tool Examples**
- JMeter / Gatling, LoadRunner / NeoLoad, Prometheus + Grafana, Dynatrace (APM), Valgrind (Dynamic Analysis)



16. Other Tools

- Some tools are focused on particular applications
 - Web-based performance testing tools
 - Language-specific static analysis tools
 - Security test tools
- Some target specific application areas like embedded systems
- Of course, testers also use spreadsheets (lots of them!) and databases
- Many developers use debugging tools
- **SOTA Tool Examples**
- Lighthouse (Web Performance), VectorCAST (Embedded), DeepChecks / IBM Adversarial Robustness Toolbox (AI Testing), GDB / LLDB (Debugging), Excel / Google Sheets



Terms to Remember

Configuration management tool

Coverage tool

Debugging tool

Dynamic analysis tool

Incident management tool

Load testing tool

Modelling tool

Monitoring tool

Performance testing tool

Probe effect

Requirements management tool

Review tool



Terms to Remember

Security tool

Static analysis tool

Stress testing tool

Test comparator

Test data preparation tool

Test design tool

Test harness

Test execution tool

Test management tool

Unit test framework tool

Test Engineering Foundation



Essential Knowledge for Test Professionals

Chapter 6: Tool Support for Testing

Section 2:
Effective use of tools, potential
benefits and risks



Effective Use of Tools

- Key concepts
 - Different scripting techniques for test execution tools, including data driven and keyword driven
 - Potential benefits and risks of test automation
- Terms to remember



Automation Opportunities and Risks

Opportunities:

- Reduced repetitive work
- Improved consistency and repeatability
- Objective assessment (especially coverage, performance, etc.)
- Simplified reporting

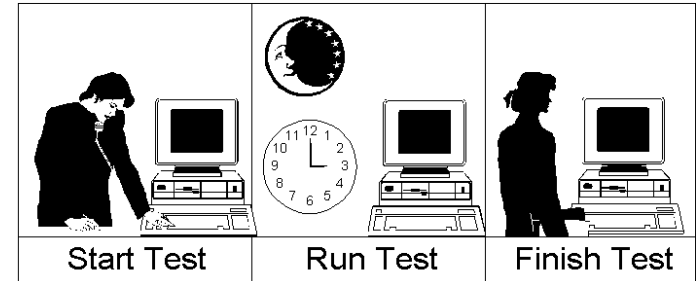
Risks:

- Unrealistic expectations
- Underestimating the time, cost and effort of development, execution, and maintenance
- Use of the tool for unsuitable tests

Automated Testing Theory and Practice



- Test automation is typically software development
 - A non-trivial process requiring significant time, skill, money
 - Automation ROI typically takes longer than one project
 - Exception: simple API-based unit, component, integration tests
 - Exception: Some non-functional tests
- Usually, what's automated is only test execution, not results analysis
 - Automation can complicate analysis (and increase number) of failed tests



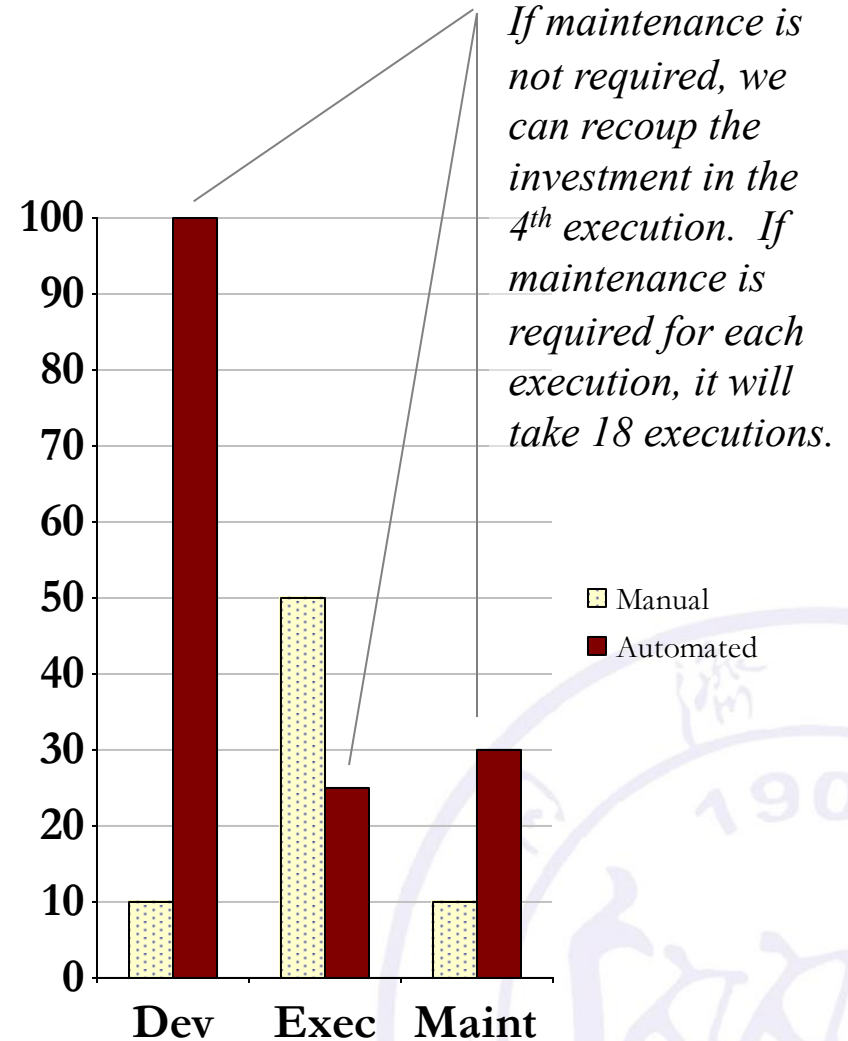
If you focus just on test execution, test automation looks very efficient, but don't forget that the automated tests had to be automated by someone to be automated.



Intelligent automation of appropriate testing tasks is a good idea.

Test Automation ROI

- A return on most test automation efforts requires:
 - Sufficiently high regression risks to justify repeating tests
 - A test system design that keeps the costs of running and maintaining the automated tests well below the cost of equivalent manual tests
 - A relatively stable system
- Must recoup high up-front test development costs over multiple projects





Choosing Manual or Automated Testing

- Tests well-suited for manual testing
 - Operations and maintenance
 - Configuration and compatibility
 - Error handling and recovery
 - Localization
 - Usability
 - Installation and setup
 - Documentation and help
- Manual, automated, or combined
 - Functional
 - Use cases (user scenarios)
 - User interface
 - Date and time handling
- Tests well-suited for automated testing
 - Regression and confirmation
 - Monkey (or random)
 - Load, volume, and capacity
 - Performance and reliability
 - Standards compliance
 - White box, especially API-based unit, component, integration
 - Static complexity and code analysis

Inappropriate manual testing misleads people about test coverage
Inappropriate automation usually fails
Some tests mix techniques

Tips for Success with Test Execution Tools



- Capture-replay seems to lower development costs, but execution and maintenance costs for large numbers of tests are high
- Data-driven tests separate inputs (the data) from procedures (scripts) that use the data
 - Non-automation experts can create test data
- Keyword-driven tests use keywords (or action words) in a file read by the script
 - Non-automation experts can create keyword files
- Automation experts create the scripts

Tips for Success with Other Tools

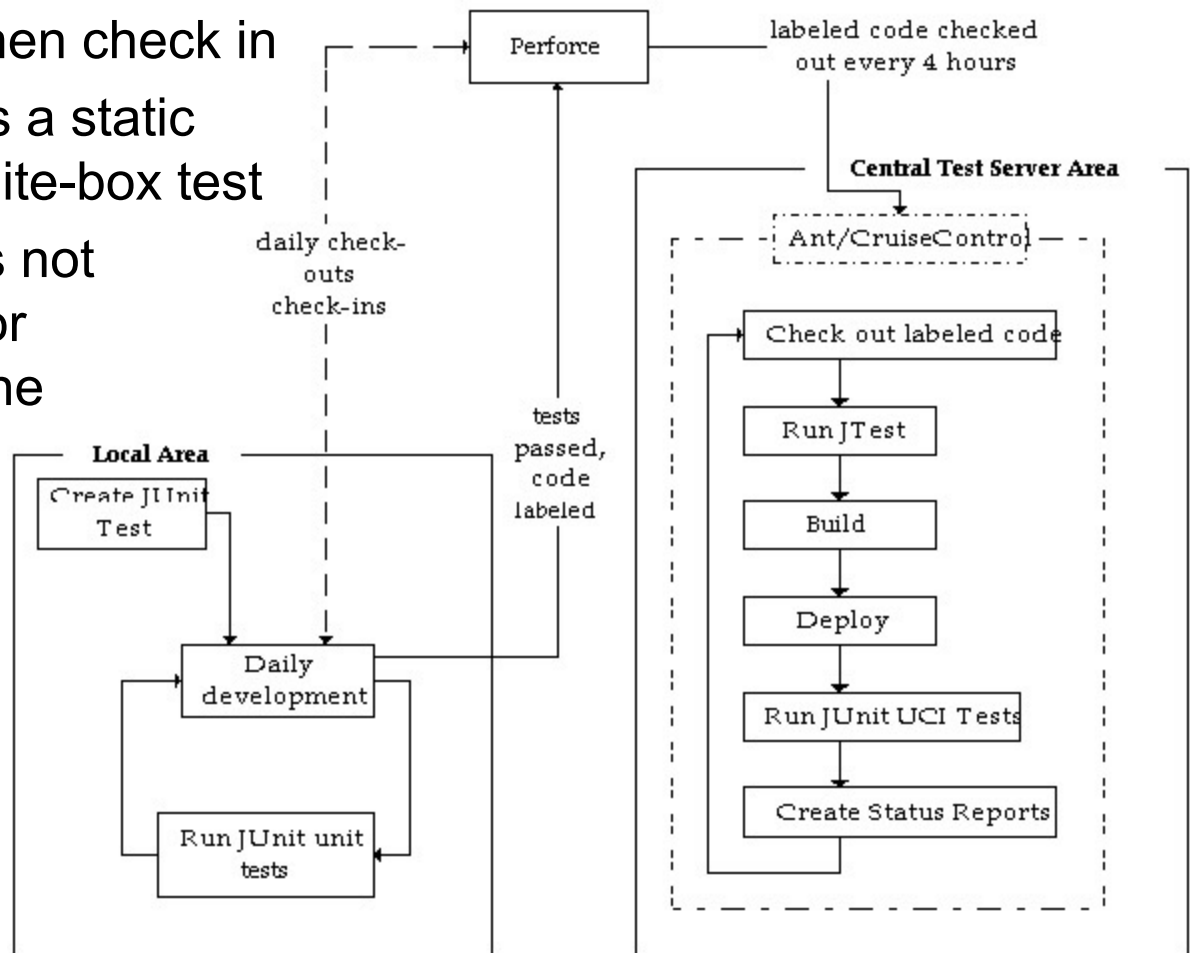


- Performance testing tools require expertise in performance to design the tests and interpret the results
- Static analysis tools applied to source code can enforce coding standards, but a phased roll-out for existing code is needed to manage the volume of errors found
- Test management tools often have poor reporting facilities, so exporting data to a spreadsheet is usually the best way to produce the desired reports



Static and White-Box Case Study

- Programmers code, test units in their local test area, then check in
- Each check-in triggers a static test, a build, and a white-box test
- While this approach is not nearly as glamorous or widely-marketing as the GUI techniques, it is cheap and quick
 - We did this in six weeks with open-source tools



See "Mission Made Possible,"
www.rbc-us.com. I thank my
colleague and coauthor on this
article, Greg Kubackowski



Terms to Remember

Data-driven testing

Keyword-driven testing

Scripting language

Test Engineering Foundation



Essential Knowledge for Test Professionals

Chapter 6: Tool Support for Testing

Section 2:
Exercise





Exercise: Omninet Test Tools

- Based on the discussion in this section, what kinds of test automation (if any) do you think would be successful for testing the kiosk?
- Based on the discussion in this section, what kinds of test automation (if any) do you think would be successful for testing the call center?
- Would you suggest automating the end-to-end testing (kiosk to call center/call center to kiosk), and, if so, how?
- Discuss.

Test Engineering Foundation



Essential Knowledge for Test Professionals

Chapter 6: Tool Support for Testing

Section 3:
Introducing a tool into an
organization





Introducing a Tool

- Key concepts
 - Introducing a tool into an organization
 - The goals of a proof-of-concept for tool evaluation
 - Factors required for good tool support
- Terms to remember

Knocking Down Barriers to Automation



- The following problems should be rectified before starting an automation effort
 - Chaotic, reactive, time-crunched test process...get the manual testing process under control first, then automate
 - Insufficiently skilled staff...hire or train
 - Unstable, rapidly changing system...stabilize the system—especially at tested interfaces like GUI and APIs—first
 - Unrealistic management expectations...effectively communicate automation benefits, limitations, business case (ROI), and costs
 - No appropriate tool or practical oracle available for the important quality risks...build own tool or oracle—or wait

When these problems remain, automation usually fails, creating organizational resistance to subsequent automation projects

Objectives of an Automation Pilot Project



- Learn more about the tool and how to use it
- Adapt the tool and/or processes and practices to fit organization and systems
- Standardize ways of using, managing, storing and maintaining the tool and the test assets
- Assess the return on investment

Success Factors for Tool Deployment



- Deploy the tool to the rest of the organization incrementally
- Adapt and improve the test processes to fit use of the tool
- Provide training, coaching , and mentoring for new users
- Define tool usage guidelines
- Learn ways to improve use of the tool continuously
- Monitor tool use and benefits



Craig's Test Tool Traps

- No clear strategy
- Unrealistic expectations
- Lack of stakeholder buy-in
- Inadequate or poor quality tool training
- Automating wrong thing (e.g., wrong tests)
- Choosing wrong tool
- Usability problems with tool
- Choosing wrong vendor
- Unstable system under test
- Doing too much, too soon
- Underestimating time and resources needed
- Inadequate or unique test environment
- Right approach, wrong time
- Excessive cost

*From Rick Craig and Stefan Jaskiel's
Systematic Software Testing, Chapter 6*

Test Engineering Foundation



Essential Knowledge for Test Professionals

Chapter 6: Tool Support for Testing

Section 3:
Exercise



Exercise: Omninet Test Tool

Introduction



- Would you use the call center or the kiosk as a pilot for automation?
- What barriers would you need to overcome to pilot the tool against the call center or kiosk?
- What test tool traps would you need to avoid?
- Discuss.



上海市嘉定区曹安公路4800号，同济大学嘉定校区软件学院